

Each section states a decision, the formal result it rests on, the citation, and the measurement from this project that confirms or contradicts it. Where a decision is only partly defensible, that is said plainly rather than dressed up.

Companion to `docs/METHODOLOGY.md`, which covers the experimental design (paired comparison, factorial vs OFAT, self-play). This file covers the decisions made after that document was written.

Decision. Every behavioural statistic is computed over games passing `engine/quality.js`. That is 1,865 of 14,794 records — 13%.

The formal point. This is not a sample-size trade-off, it is a **target-population** question. An estimator is unbiased *for the population it samples from*. The quantity we want is "how do humans play Champions"; the raw store estimates "how do humans-and-bots-and-forfeiterers play". Adding contaminated records does not reduce the variance of the estimate we want — it estimates a different parameter with more precision. Precision about the wrong quantity is not an improvement.

The circularity is the sharper problem. Self-play exists to produce games that resemble *human* play. Measuring our bots against a corpus that is 63% other people's bots and calling the difference "realism" evaluates the model against a population it was never meant to imitate.

Measured, and it cuts both ways. The bot games are *not* weak:

	clean humans	ladder bots	our self-play
super effective	21.4%	24.2%	11.2%
moves that failed	2.5%	2.8%	8.6%
turns per game	8.4	6.2	10.8

So contamination shifted the target by ~3 points while our own gaps are 10–18. **Direction of every conclusion survived; precision did not.** Reported here because the honest reading is "this mattered less than feared", not "this vindicated the filter".

The residual worry, stated. 1,865 games is thin, and replays are uploaded *voluntarily* — a self-selected sample, so even the clean subset is not a random draw from the ladder. This is why anything about team composition is taken from Smogon (1,163,315 battles, computed server-side over every team played) rather than from our store. Our store is used only for what Smogon does not publish: per-turn behaviour.

Decision. In `set_space.js`, a move Smogon reports on ≥50% of sets is treated as fixed when deciding which slots a build experiment should vary. It is **not** force-included during set generation — see the limit below, which is the more instructive half of this section.

The formal result. Under 0–1 loss on a discrete outcome, the Bayes-optimal decision is the **mode of the conditional distribution**, $\operatorname{argmax}_a p(Y = a \mid X)$ — not a draw from that distribution ([Stephens, Bayes decision rule for prediction](#)). Sampling in proportion to the probabilities is **probability matching**, and it is strictly suboptimal for accuracy; the Bayes rule pushes decisions toward 0 or 1 rather than tracking the base rate ([Bayesian optimal treatment regimes](#), [arXiv 1809.06679](#)).

Specialised to one binary slot: always-including a move that truly sits on a fraction p of sets reproduces reality on p of sets. Sampling it with probability p reproduces reality on $p^2 + (1-p)^2$. The difference is

$$p - (p^2 + (1-p)^2) = (2p - 1)(1 - p)$$

positive exactly when $p > \frac{1}{2}$. **The crossover is arithmetic, not a tuned threshold** — which is what makes it admissible under S12/S13.

THE HONEST LIMIT — AND IT IS LARGER THAN THIS SECTION FIRST CLAIMED.

This was implemented, measured, and reverted the same hour. Forcing every move above 50% put all of them on 100% of generated sets and drove Kingambit's Low Kick from 47.8% to **zero**, Swords Dance to **zero**. Mean deviation from Smogon went from 2.6 points under to 17.5 points over. The set space collapsed to one build per species.

The error was applying the right theorem to the wrong objective. Two different goals are in play and they have *opposite* optima:

goal	optimal rule
guess ONE opponent's set as accurately as possible	take the mode (Bayes, 0-1 loss)
generate a CORPUS whose sets are distributed like reality	sample from the distribution

Probability matching is suboptimal for per-instance accuracy and *correct* for distributional fidelity — a generated corpus is closest to the truth in distribution when it is drawn from the truth. Set generation exists to populate a corpus, so it wants the second rule. XATU, which predicts a specific opponent's set from partial revelation, wants the first.

So the 50% rule stands where it was derived — `set_space.js`, choosing which slots a build experiment should hold fixed — and does **not** govern set generation. The earlier text in this section implied otherwise and was wrong.

The real defect, correctly sized. Our locked moves appear 2.6 points *less* often than Smogon says (Farigiraf 5.8, Kingambit 5.5, Sinistcha 0.3) — a small, systematic under-production, not a missing rule. Its cause is the sequential weighted draw: once common moves are taken, the four-slot constraint forces rare ones onto the set. The principled fix is to sample each move independently at its Smogon *set-rate* and repair to exactly four, which targets the marginals directly. Open, and deliberately not fixed by forcing.

Decision. Report, and refuse to chase, the fraction of real sets containing a move Smogon does not list individually.

The formal frame. This is the **unseen-species problem**, posed by Corbet to Fisher in the 1940s and solved by Good and Turing in 1953: estimate the total probability of outcomes absent from the sample ([Good–Turing overview](#); [Generalized Good–Turing](#), *JASA* 118(543)). The quantity is the **missing mass**, and estimating it is a standard, well-posed problem with known minimax risk ([arXiv 1705.05006](#)).

Our situation is the easy case. Smogon *publishes* the missing mass directly as the "Other" bucket, so no estimator is needed — the tail probability is given. Converting per-slot mass to per-set probability uses only the fact that a set holds four moves:

$$P(\text{set contains an unlisted move}) = 1 - (1 - \text{other}/400)^4 \approx 17\%$$

Median 17% across 283 species, worst 19%, Kingambit 3%.

Why this framing matters practically. Missing mass is a property of the *sampling process*, not a defect in our pipeline. No amount of work on our side recovers it, because the information was discarded before publication. Naming it correctly is what licenses the decision to stop — and this framing is being used the same way in ML deployment-coverage work ([Blind-Spot Mass](#), [arXiv 2604.05057](#)).

Caveat. Independence across the four slots is assumed and is certainly false — movesets are correlated. The figure is therefore approximate, and should be read as "about one set in six", not as a calibrated number.

Decision. In `build_lab`, the Pokémon whose build is being measured selects uniformly over its legal moves. Opponents keep the usage-weighted behaviour clone.

The problem it solves. Usage priors are renormalised over whichever four moves a build carries, so the *amount of measurement each arm receives depends on the arm*. Measured on Garchomp: the tested slot takes 29.4% of the pilot's decisions as Earthquake and 25.2% as Stomping Tantrum — **0.86×**. A rare move gets less opportunity to demonstrate its effect, and rarity is the treatment under study.

This is treatment-correlated measurement intensity — a confound, not noise. It biases toward builds made of common moves, which is circular, since commonality is the hypothesis being tested rather than a fact to be assumed. Equal allocation across arms is the standard remedy: with equal variances, balanced allocation minimises the variance of pairwise contrasts, which is why balance is the default in comparative designs ([Czitolom, One-Factor-at-a-Time Versus Designed Experiments](#)).

THE FLAW, which is real and was pointed out immediately. Uniform allocation is unbiased across moves **only if the moves are exchangeable in time**. Setup moves are not: Swords Dance, Trick Room and Tailwind have value only when played *before* the attacks they enable. A uniform pilot clicks Swords Dance on the last turn, or twice, and a build containing it is penalised for reasons unrelated to the build.

So the change removes a bias in *how much* each move is measured and leaves a bias in *when*. That is an improvement — the first is proportional to the treatment, the second is a property of the pilot shared by all arms carrying setup — but it is not a solution, and `build_lab` currently understates setup builds.

The three options, ranked.

1. **Hand-code "setup first."** Rejected: asserted domain logic, and the list never ends.
 2. **Condition the move prior on turn index**, measured from clean games. Derived rather than typed, and fixes most of the effect. Defensible as an interim.
 3. **A board-conditioned policy.** The right answer. Clicking setup when the turn is free is not a rule about setup moves; it is what reading the board means. Everything above is scaffolding until it exists.
-

Decision. Per-species build structure ("freedom") is computed as $(400 - \text{sum of the top four moves}) / 100$ rather than from a usage threshold.

Why it is not a modelling choice. Smogon's move percentages are shares of *sets*, and every set holds exactly four moves, so the listed percentages plus "Other" sum to 400 by construction. The quantity is therefore a **compositional constraint**, not an estimate: the residual after the four most common moves is, by arithmetic, the expected number of slots differing from the standard build.

Garchomp 0.71, Farigiraf 1.47, Kingambit 0.59. Across 259 species with 2,000+ teams the four most common moves account for 68% of all move slots played.

This replaced a hand-written `LOCK_AT = 85`, which was both invented and wrong — it classified Garchomp's Earthquake (76.9%) as a free choice.

- **Every current `build_lab` win rate is conditioned on a pilot that does not read the board** (super-effective 11.2% against a human 21.4%; failed moves 8.6% against 2.5%). No statistical correction repairs that, and it makes each result provisional.
- **External validity is untested.** `build_lab` measures one host team against 40 opponents and does not currently state that scope in its output.
- **24 engine tools** still read the ladder store without a clean filter or a declared reason. Their published numbers are unverified. `engine/selftest.js` fails on this by design.